19-Day Full Stack Training Program

Angular + .NET (8 Hours Per Day)

Course Overview

This intensive 19-day Full Stack Development program is designed to take participants from core C# programming fundamentals to building secure, real-time, enterprise-ready web applications using ASP.NET Core and Angular. The course follows a structured architectural approach covering N-Tier design, RESTful APIs, middleware pipeline, dependency injection, JWT authentication, and modern frontend development with Angular, RxJS, and SignalR integration.

Participants will progressively build backend services, integrate frontend applications, and implement real-time communication, culminating in a full-stack capstone review and assessment.

Duration

- Total Duration: 19 Days
- Recommended Duration: 25 Days (Recommended for better room for practice, deeper explanations, and structured implementation)
    - C# Foundations: 5 + 2
    - .NET Core & Web API: 5 + 2
    - Angular: 6 + 1
    - Project & Integration: 1 + 1
- Hours per Day: 8 Hours
- Total Learning Hours: 152 Hours (200 Hours in 25-Day format)
- Format: Instructor-led with hands-on implementation

Prerequisites

Participants should have: - Basic understanding of programming concepts (any language) - Familiarity with basic HTML & CSS - Logical reasoning and problem-solving skills

No prior experience in C#, .NET, or Angular is required.

System Requirements

- Windows / macOS / Linux (64-bit)
- Minimum 8 GB RAM (16 GB recommended)
- Latest .NET SDK installed
- Visual Studio / VS Code
- Node.js (LTS version)
- Angular CLI
- SQL Server / SQL Express
- Postman or Thunder Client
- Modern browser (Chrome recommended)

Final Course Outline

WEEK 1 – C# Foundations (5 Days)

Day 1 – C# Programming Fundamentals

- .NET ecosystem overview
- Setting up SDK & IDE
- C# syntax and structure
- Variables, data types, operators
- Control statements (if, switch, loops)
- Methods and parameter passing
- Hands-on exercises

Day 2 – Core OOP in C

- Classes and Objects
- Constructors & object initialization
- Encapsulation
- Inheritance
- Polymorphism
- Abstraction
- Access modifiers
- OOP design exercises

Day 3 – Advanced OOP + SOLID Principles

- Interfaces and abstraction in practice

- Abstract classes vs interfaces
- SOLID principles (deep dive with examples)
- Real-world design problems
- Refactoring toward SOLID
- Introduction to testable design

Day 4 – Collections, Generics, LINQ + Unit Testing (Part 1)

- Collections (List, Dictionary, HashSet)
- Generics
- IEnumerable & LINQ fundamentals
- Lambda expressions
- Introduction to unit testing
- xUnit / NUnit setup
- Writing first test cases

Day 5 – Async Programming + Unit Testing (Part 2)

- Thread basics
- Task and async/await
- Asynchronous patterns
- Testing async methods
- Assertions & test organization
- Test-driven thinking
- Refactoring safely using tests

WEEK 2 – .NET Core & Web API (5 Days)

Day 6 – Web Architecture, HTTP & N-Tier Design

- What is N-Tier architecture (Presentation, Business, Data layers)
- Role of API in modern applications
- HTTP fundamentals (request/response, verbs, headers, status codes)
- REST principles
- Designing RESTful URLs (resource-based routing)
- Understanding JSON and content negotiation
- API project structure overview (no controllers yet)

Day 7 – ASP.NET Core Pipeline, Middleware & DI (No Controllers Yet)

- Introduction to ASP.NET Core runtime
- Request processing pipeline

- Built-in middleware
- Creating custom middleware
- IServiceCollection & service registration
- Service lifetimes (Transient, Scoped, Singleton)
- Injecting services into middleware
- Configuration & environment setup

Day 8 – Introducing Controllers & API Endpoints

- Adding MVC / Controller middleware
- Creating controllers and action methods
- Routing (attribute routing & conventional routing)
- Model binding basics
- Returning IActionResult properly
- Testing endpoints using Swagger
- Minimal APIs (architectural comparison)

Day 9 – Data Modeling, EF Core & Layered Implementation

- Designing domain models
- DbContext and DbSet
- Code First approach
- Migrations
- Repository pattern
- Service layer abstraction (Controller → Service → Repository → DbContext)
- Why controllers should not talk directly to DbContext
- Basic CRUD implementation
- Testing APIs using Postman / Thunder Client

Day 10 – Validation, Filters, Exception Handling & JWT Security

- Data annotations & model validation
- Custom validation logic
- Action filters
- Exception filters
- Global exception handling strategy
- Consistent API response structure
- Authentication vs Authorization
- JWT fundamentals
- Generating tokens and securing endpoints
- Testing secured APIs in Postman

WEEK 3 – JavaScript, TypeScript & Angular (8 Days)

Day 11 – JavaScript Fundamentals

- JavaScript basics
- Variables and scope
- Functions
- Arrays and objects
- ES6 features (let/const, arrow functions, spread)
- Promises & async/await

Day 12 – TypeScript for Angular

- Why TypeScript
- Types and interfaces
- Classes and access modifiers
- Generics in TypeScript
- Modules and configuration
- Compiling TS to JS

Angular (6 Days)

Day 13 – Angular Fundamentals

- SPA architecture
- Angular CLI
- Project structure
- Components and templates
- Data binding (interpolation, property, event)

Day 14 – Directives & Component Communication

- Structural directives
- Attribute directives
- @Input and @Output
- Lifecycle hooks
- Reusable component design

Day 15 – Routing & Guards (Moved Earlier)

- Angular routing fundamentals
- Router configuration

- Route parameters
- Child routes
- Lazy loading modules
- Route guards (CanActivate)
- Using in-memory data for navigation demos
- Basic navigation security concepts

Day 16 – Forms & Validation

- Template-driven forms
- Reactive forms
- FormBuilder
- Custom validators
- Dynamic forms
- Connecting forms with in-memory data

Day 17 – Services, HTTP & RxJS (After Angular Basics)

- Angular services
- Dependency Injection in Angular
- Introducing service-based state (in-memory first)
- HttpClient integration
- Observables fundamentals (after basics are clear)
- Understanding subscribe vs async pipe
- Interceptors
- Error handling strategies

Day 18 – RxJS, SignalR & Secure Integration

- Observable fundamentals (next, error, complete)
- Observable vs Promise
- Async pipe vs manual subscription
- Introduction to Subject & BehaviorSubject
- Simple shared-state example using BehaviorSubject
- WebSockets fundamentals
- Introduction to SignalR (ASP.NET Core Hub)
- Connecting Angular client to SignalR
- Connecting Angular to JWT-secured .NET API
- HttpInterceptor for token handling
- Handling 401/403 & Guard integration

WEEK 4 – Capstone

Day 19 – Capstone Project Review & Assessment

- Full stack project presentation
- Code walkthrough
- Architecture discussion
- Best practices evaluation
- Individual technical assessment
- Feedback & improvement roadmap

Outcome of the Program

Participants will be able to: - Write production-ready C# applications with unit testing - Build secure RESTful APIs using ASP.NET Core - Apply middleware and dependency injection properly - Test APIs using Postman / Thunder Client - Develop Angular applications using TypeScript - Integrate frontend and backend securely - Implement authentication and role-based access - Deliver a complete full stack solution